



YAŞAR UNIVERSITY

COMP 3328: EMBEDDED SYSTEMS PROJECT

**GAS LEAKAGE DETECTION AND AUTOMATIC
CONTROL SYSTEM**

NİSA KOCAGENİŞ

22070001037

TERM PROJECT ADVISOR: ASSIST.PROF.(PHD) ÖMER ÇETİN

ENGINEERING FACULTY

COMPUTER ENGINEERING PROGRAM

BORNOVA / İZMİR
MAY 2026

APPROVAL PAGE

I acknowledge and declare that I have personally worked in accordance with academic ethical principles during the preparation of this project. I accept and undertake that if the contrary situation is detected in any way, my project work will not be accepted and will not be evaluated, and I will be subject to sanctions announced by the university administration.

Student:

Nisa / KOCAGENİŞ

22070001037

01/05/2026

Signature:

.....

ABSTRACT

Gas Leakage Detection and Automatic Control System

KOCAGENİŞ, NİSA

COMP 3328: Embedded Systems Project Term Project,

Advisor: Assist. Prof.(PhD) Ömer ÇETİN

MAY 2026

Residential and industrial facilities are among the environments most vulnerable to gas leakage incidents, which can result in serious loss of life and property. In this study, an embedded system has been developed that continuously monitors the gas concentration in the environment and autonomously activates protective measures without requiring human intervention.

The system measures the gas level in the environment using an MQ-2 gas sensor; the analog data obtained is processed by the ATmega328P microcontroller and compared against predefined threshold values. When a gas leakage is detected, the system initiates a two-stage response protocol. In the first stage, a warning message is displayed on a 16x2 LCD screen, an LED provides a visual alert, and an audible alarm is triggered via a buzzer. In the second stage, servo motors are activated to shut off the gas valve and automatically open the ventilation system. The system has been developed using the AVR C programming language and designed and tested in the Wokwi simulation environment.

Keywords: *gas leakage detection, embedded systems, MQ-2 sensor, ATmega328P, Arduino, servo motor, AVR C, Wokwi, automatic control system*

TABLE OF CONTENTS

APPROVAL PAGE	3
ABSTRACT	5
TABLE OF CONTENTS	7
1. CHAPTER: PROBLEM DEFINITION	1
2. CHAPTER: THE GOAL OF THE PROJECT	5
3. CHAPTER: HARDWARE	7
1. Microcontroller	7
2. Diagram Definition of Project	8
3. Other Input / Output Devices Used within the Scope of the Project	9
4. Cost of Project	10
4. CHAPTER: SOFTWARE DEVELOPMENT	11
4.1. Functions	12
4.2. Constants & Variables	13
5. CHAPTER: CODE	15
2. AVR C Code	16
3. WOKWI diagram.json File	22
4. WOKWI libraries.txt File	24
6. CHAPTER: PROJECT RUNTIME SCREENSHOT IMAGES	25
7. CHAPTER: CONCLUSIONS AND FUTURE WORK	27
8. REFERENCES	29

Nowadays, natural gas and LPG are widely used for heating and cooking purposes in both residential and industrial environments. Although these gases provide significant convenience in daily life and industrial operations, their flammable and toxic characteristics create serious safety risks in the event of leakage. A gas leak that is not detected and controlled in time may cause poisoning, fire, explosion, and severe loss of life and property. In particular, carbon monoxide gas is known as the “silent killer” because it is colorless and odorless, making it difficult for people to notice without a detection system.

Most existing gas alarm systems only warn the user through audible or visual alerts. However, warning alone may not be sufficient in emergency situations. If the user is asleep, panicked, away from home, or unable to react quickly, the danger may continue to grow. This problem becomes even more critical in industrial areas such as factories, warehouses, industrial kitchens, and mines, where gas leaks can affect many people and may trigger larger accidents such as fires and explosions. Therefore, there is a need for a system that not only detects gas leakage but also responds to it automatically.

The frequency and impact of CO poisoning and gas leakage incidents may vary between countries due to socioeconomic conditions, heating habits, building structures, and climate differences. Turkey presents an important example in this regard, especially because natural gas and LPG are commonly used in homes and workplaces. Table 1.1 presents the distribution of CO poisoning cases and deaths by province in Turkey

	İlin Adı	Vaka Sayısı	Ölüm Sayısı
1	İstanbul	1.500 (%14,7)	1 (%2,5)
2	Bursa	1.134 (%11,1)	18 (%46,1)
3	Ankara	894 (%8,8)	2 (%5,1)
4	İzmir	495 (%4,8)	0
5	Eskişehir	466 (%4,5)	0
6	Kocaeli	462 (%4,5)	3 (%7,6)
7	Kayseri	359 (%3,5)	0
8	Denizli	295 (%2,9)	1 (%2,5)
9	Sivas	279 (%2,7)	0
10	Kilis	246 (%2,4)	3 (%7,6)
11	Adıyaman	216 (%2,1)	0
12	Kahramanmaraş	212 (%2,0)	0
13	Niğde	183 (%1,8)	3 (%7,6)
14	Uşak	175 (%1,7)	1 (%2,5)
15	Konya	171 (%1,6)	0
16	Hatay	157 (%1,5)	0
17	Karabük	145 (%1,4)	0
18	Gaziantep	139 (%1,3)	0
19	Elazığ	105 (%1,0)	0
20	Afyon	105 (%1,0)	0
	Türkiye Geneli	10.154 (%100)	39 (%100)

Table 1.1 — Distribution of CO poisoning cases by province

As shown in Table 1.1, a total of 10,154 CO poisoning cases and 39 deaths were recorded across Turkey. Istanbul, Bursa, and Ankara stand out in terms of the number of cases, while Bursa draws attention with its high death rate. These data show that CO poisoning and gas leakage risks are not only individual safety problems but also important public health and occupational safety issues.

In addition, the effect of CO gas on human health changes rapidly depending on the gas concentration in the environment. As the gas level increases, symptoms appear in a shorter period of time, and at high concentrations, the risk of death may occur within minutes. Table 1.2 shows poisoning symptoms and effects depending on CO concentration.

CO Ratio (%)	CO Concentration (ppm)	Symptoms and Effects
0.02	200	Headache within 2-3 hours
0.04	400	Headache within 1-2 hours, severe headache within 3 hours
0.08	800	Dizziness, nausea, and cramps in the limbs within 45 minutes; unconsciousness within 2 hours
0.16	1600	Headache and dizziness within 20 minutes; death within 2 hours
0.32	3200	Headache, dizziness, and nausea within 5-10 minutes; death within 30 minutes
0.64	6400	Headache, dizziness, and nausea within 1-2 minutes; death within 10-15 minutes
1.28	12800	Death within 1-2 minutes

Table 1.2. Poisoning Symptoms and Effects Depending on CO Concentration in the Environment

As shown in Table 1.2, 200 ppm of CO in the environment may cause headache within a few hours, while 1600 ppm can create a risk of death within two hours. At higher levels, the time required for fatal effects decreases to only a few minutes. Therefore, warning alone is not sufficient in gas leakage systems. The system must continuously monitor gas concentration, activate alarms quickly when the threshold value is exceeded, and initiate physical intervention.

Based on this need, this project proposes a Gas Leakage Detection and Automatic Control System that can be used in both residential and industrial environments. The system is designed as an embedded safety solution using an Arduino Uno board with the ATmega328P microcontroller. It continuously monitors the gas concentration in the environment through an MQ-2 gas sensor. The analog data received from the sensor is processed by the microcontroller and compared with predefined safety threshold values.

When the gas level exceeds the safety threshold, the system activates a two-stage response mechanism. In the first stage, the user is warned through a 16x2 LCD screen, LED indicator, and buzzer alarm. The LCD displays warning messages such as “Gas Leakage Detected,” while the LED and buzzer provide visual and audible alerts. In the second stage, the system performs autonomous physical intervention. One servo motor closes the gas valve, and another servo motor opens a window or ventilation mechanism to reduce the gas concentration in the environment. If required, a relay module can also be used to control external high-power valve or ventilation systems.

The main advantage of this project is that it reduces dependency on human intervention during emergency situations. Users can access the system status directly through the LCD screen and understand whether the environment is safe or dangerous. Since the system operates continuously and automatically, it provides ease of use for homes, factories, warehouses, industrial kitchens, and similar high-risk areas. The project will be developed using AVR C programming language and tested in the Wokwi simulation environment, where sensor readings, alarm mechanisms, LCD messages, servo motor movements, and overall system behavior will be verified under different gas leakage scenarios.

The main goal of this project is to develop an embedded system that detects gas leakage at an early stage, warns the user quickly, and automatically takes safety actions without requiring human intervention in dangerous situations. In gas leakage incidents, giving only an alarm may not always be sufficient. If the user is not present in the environment, asleep, panicked, or unable to respond quickly, the danger may continue to increase. Therefore, this project is designed not only as a warning system but also as an autonomous safety solution that performs physical intervention.

During the development process of the project, the ATmega328P microcontroller on the Arduino Uno board will be used. The gas concentration in the environment will be continuously measured using an MQ-2 gas sensor, and the analog data received from the sensor will be processed by the microcontroller and compared with predefined threshold values. The system software will be developed using the AVR C programming language, and the design will be tested in the Wokwi simulation environment. During the simulation process, sensor readings, LCD messages, LED and buzzer warnings, servo motor movements, and the overall system response will be checked under different gas leakage scenarios.

This project is planned to be used in both residential and industrial areas. Homes, apartments, factories, warehouses, industrial kitchens, laboratories, mines, and similar environments where natural gas or LPG is used are among the intended usage areas of the system. In this respect, the project provides a solution that can improve home safety for individual users and also contribute to occupational safety in workplaces and industrial facilities.

The main features intended to be added to the system are gas concentration detection, threshold value control, audible and visual warning, displaying system status on an LCD screen, and automatic physical intervention in case of danger. When the gas level exceeds the predefined safety threshold, a warning message will be displayed on the LCD screen, the LED will turn on, and the buzzer will generate

an audible alarm. At the same time, one servo motor will close the gas valve, while the other servo motor will open the window or ventilation mechanism. If required, a relay module can also be used to control higher-power external valve or ventilation systems.

The most important ease of use provided by the project is that the system operates continuously and does not require manual monitoring by the user. Without requiring the user to constantly check the gas level, the system will automatically monitor the environment and respond within seconds when a dangerous situation occurs. Users will be able to access the system directly through the 16x2 LCD screen and instantly see whether the environment is safe or dangerous. In addition, the LED indicator and buzzer alarm will provide visual and audible warnings even when the user is not looking at the screen.

In conclusion, the goal of this project is to develop a practical, reliable, and easy-to-use embedded system that provides early detection, effective warning, and autonomous intervention against gas leakage risks. The system aims to increase life and property safety by reducing dependency on human intervention, especially in emergency situations.

1. Microcontroller

In this project, the **ATmega328P microcontroller** on the Arduino Uno board will be used. The ATmega328P has sufficient hardware features for the gas leakage detection and automatic intervention system, such as reading sensor data, controlling warning components, and operating servo motors.

Since analog data will be received from the MQ-2 gas sensor in the project, the microcontroller needs to have analog input and ADC support. Thanks to the built-in ADC unit of the ATmega328P, the analog gas concentration value coming from the sensor can be read and compared with the threshold value defined in the software. In this way, the system can decide whether the gas level is normal or dangerous.

In addition, a 16x2 LCD screen, LED, buzzer, two servo motors, and a relay module if required will be used in the system. The ATmega328P has enough digital input/output pins to control these components. The PWM signals required for controlling the servo motors can also be generated by the ATmega328P. This feature allows the servo motors to move to the correct angles for closing the gas valve and opening the window or ventilation mechanism.

Another reason for choosing the ATmega328P is that it is based on the AVR architecture and allows low-level hardware control using the AVR C programming language. In this way, sensor reading, threshold value control, alarm mechanisms, and motor movements can be programmed more efficiently and in a controlled manner. In addition, the support of the Arduino Uno board in the Wokwi simulation environment makes the design and testing stages of the project easier.

High processing power, wireless communication, or advanced multimedia features are not required for this project. Therefore, the ATmega328P is a suitable microcontroller option because it is low-cost, easy to implement, provides sufficient input/output support, and meets the needs of the project.

2. Diagram Definition of Project

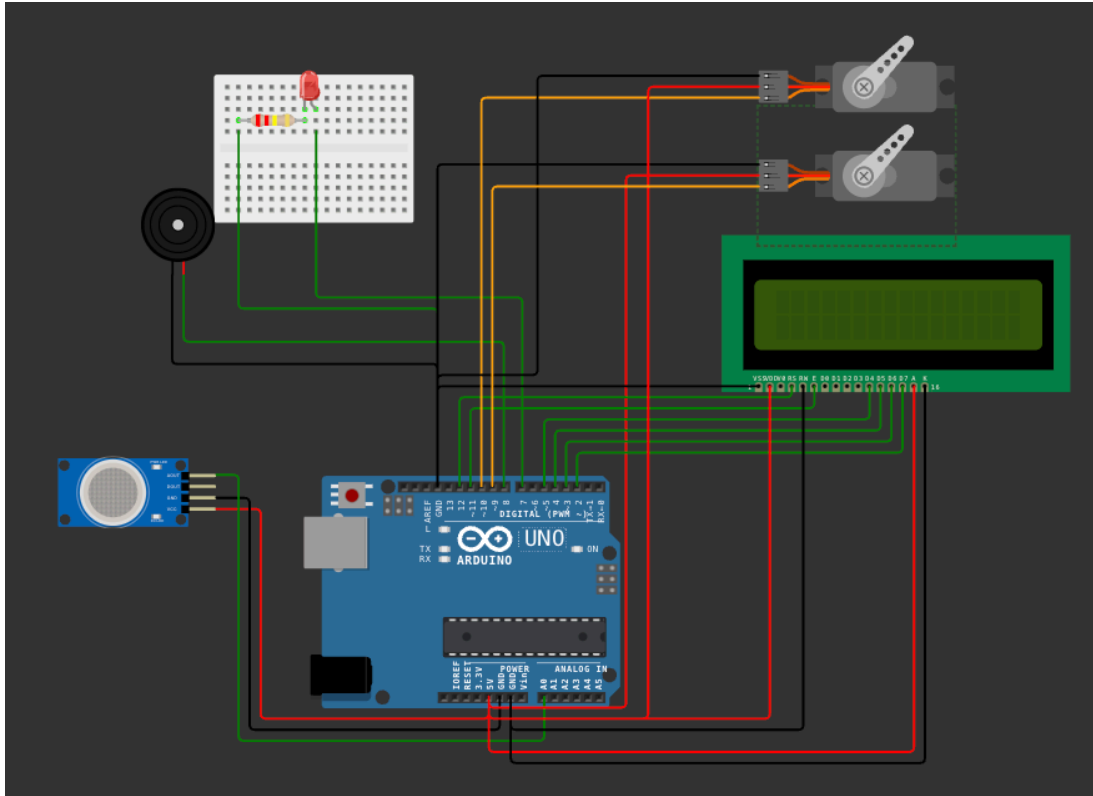


Figure 2.1. Image Description

Source: Author

This part of the project presents the circuit design of the Gas Leakage Detection and Automatic Control System developed in the simulation environment. The circuit includes an Arduino Uno board, MQ-2 gas sensor, 16x2 LCD display, LED, buzzer, and two servo motors. The MQ-2 sensor is used to detect the gas concentration in the environment and send analog data to the microcontroller. When the gas level exceeds the predefined threshold value, the system activates the buzzer and LED to warn the user. At the same time, the servo motors are triggered to simulate closing the gas valve and opening the window or ventilation mechanism. The LCD screen is used to display the current system status and warning messages to the user.

3. Other Input / Output Devices Used within the Scope of the Project

All display, output, and sensor hardware planned to be used within the scope of the project is listed below. In this project, no separate user input device such as a keypad, button, joystick, or switch is used. The main input data of the system is obtained from the MQ-2 gas sensor. The sensor measures the gas concentration in the environment and sends analog data to the ATmega328P microcontroller. The microcontroller processes this data and controls the display, alarm, and intervention components according to the predefined threshold value.

Type	I/O Device	Purpose of usage
<i>Display</i>	<i>LED/ RGB LED</i>	<i>It is planned to be used to visually indicate normal and dangerous system states.</i>
<i>Display</i>	<i>LCD (16x2)</i>	<i>It is planned to be used to display system status and warning messages to the user.</i>
<i>Sensor</i>	<i>MQ-2 Gas Sensor</i>	It is planned to be used to detect gas concentration in the environment and provide analog data to the microcontroller.
<i>Output</i>	<i>Buzzer</i>	It is planned to be used to provide an audible alarm when gas leakage is detected.
<i>Output</i>	<i>Servo(Gas Valve)</i>	<i>It is planned to be used to automatically shut off the gas valve in case of gas leakage.</i>

<i>Output</i>	<i>Servo Motor (Window/Ventilation)</i>	It is planned to be used to open the window or ventilation system for gas
<i>Output</i>	<i>Relay Module</i>	It is planned to be used to control external high-power valve or

4. Cost of Project

Component	Quantity	Unit Price (TL)	Total (TL)	Source
Arduino UNO (ATmega328P)	1	~250	250	robotistan.com
MQ-2 Gas Sensor	1	~92	92	robotsepeti.com
LCD Display 16x2	1	~60	60	robotistan.com
SG90 Servo Motor	2	~45	90	robotistan.com
Buzzer	1	~10	10	robotistan.com
LED (Red/Green)	2	~2	4	robotistan.com
Relay Module	1	~35	35	robotistan.com
Breadboard + Jumper Cables	1 set	~50	50	robotistan.com
TOTAL			~591 TL	

4.

CHAPTER: SOFTWARE DEVELOPMENT

In this project, the software development process focuses on reading the gas concentration value from the MQ-2 gas sensor, evaluating this value according to a predefined threshold, and controlling the output devices based on the result. The software is developed using the AVR C programming language for the ATmega328P microcontroller on the Arduino Uno board. Instead of using Arduino-specific functions, the project code directly uses AVR registers such as DDRB, DDRD, PORTB, PORTD, ADMUX, ADCSRA, TCCR1A, TCCR1B, OCR1A, and OCR1B.

The software development process can be examined in three main steps: hardware initialization, sensor data reading, and system response control.

In the hardware initialization step, the input and output pins are configured. The LED and buzzer pins are set as output pins. The LCD pins are also configured as output pins because the microcontroller sends command and character data to the LCD. The ADC module is initialized to read the analog value from the MQ-2 gas sensor. In addition, Timer1 is configured to generate PWM signals for the servo motors connected to Arduino D9 and D10 pins.

In the sensor data reading step, the gas sensor value is read from the A0 analog input using the ADC module of the ATmega328P. The MQ-2 sensor produces an analog voltage according to the gas concentration in the environment. This analog signal is converted into a digital value between 0 and 1023 by the ADC. The obtained value is then compared with the predefined gas threshold value.

In the system response control step, the software decides whether the environment is safe or dangerous. If the gas value is below the threshold, the system remains in safe mode. In this mode, the LED and buzzer are turned off, the gas valve servo remains open, and the window servo remains closed. The LCD displays the current gas value and a safe status message. If the gas value is equal to or higher than the threshold, the system switches to danger mode. In this mode, the LED is turned

on, the buzzer generates an audible alarm, the gas valve servo closes the valve, and the window servo opens the ventilation mechanism. The LCD displays both the gas value and a gas leakage warning message.

The software is designed to operate continuously in an infinite loop. The gas value is measured periodically, displayed on the LCD screen, and evaluated according to the threshold value. This structure allows the system to monitor the environment continuously and respond automatically when a dangerous gas level is detected.

4.1. Functions

The software consists of several functions that are used to control the LCD, ADC, servo motors, and system outputs.

The LCD functions are used to initialize the LCD, send commands, send characters, clear the screen, set the cursor position, and print text. The `lcd_enable()` function sends a short enable pulse to the LCD so that the LCD can read the data placed on its data pins. The `lcd_send()` function sends 4-bit data to the LCD data pins. The `lcd_control()` function sends control commands to the LCD, such as clearing the screen or setting the cursor position. The `lcd_character()` function sends a single character to the LCD. The `lcd_initialize()` function prepares the LCD for operation in 4-bit mode. The `lcd_clear()`, `lcd_set_cursor()`, and `lcd_print()` functions are used to display messages on the LCD in a readable format.

The ADC functions are used to read the gas sensor value. The `adc_initialize()` function activates the ADC module, selects AV_{cc} as the reference voltage, and sets the ADC prescaler. The `read_gas_sensor()` function starts an ADC conversion on the A0 channel and returns the converted digital value between 0 and 1023.

The servo functions are used to control the gas valve and window mechanisms. The `servo_initialize()` function configures Timer1 to generate PWM

signals on D9 and D10 pins. The `servo_valve_open()` and `servo_valve_close()` functions control the servo motor connected to the gas valve. The `servo_window_open()` and `servo_window_close()` functions control the servo motor connected to the window or ventilation mechanism.

The `main()` function is the main control part of the program. It initializes all required modules, starts the system in safe mode, reads the gas sensor value continuously, displays the value on the LCD, and activates the related outputs according to the gas level.

4.2. Constants & Variables

Several constants and variables are used in the software to make the code more understandable and easier to modify.

The `F_CPU` constant is defined as `16000000UL` because the ATmega328P microcontroller on the Arduino Uno board operates at 16 MHz. This value is required for the delay functions to calculate accurate time delays.

The `GAS_LIMIT` constant is defined as `400`. This value represents the threshold level used to decide whether the gas concentration is safe or dangerous. If the gas sensor value is equal to or greater than this limit, the system considers the situation as gas leakage and activates the alarm and intervention mechanisms.

Pin constants are used to define the hardware connections. The buzzer is connected to `PB0`, which corresponds to Arduino D8. The LED is connected to `PD7`, which corresponds to Arduino D7. The LCD control pins are defined as `LCD_RS` and `LCD_EN`, and the LCD data pins are defined as `LCD_D4`, `LCD_D5`, `LCD_D6`, and `LCD_D7`. These definitions make the code easier to read and modify if the pin connections are changed later.

The variable `gas_value` stores the digital value read from the MQ-2 gas sensor. This value is obtained from the ADC and is used in the comparison with the

threshold value. The variable `gas_text` is a character array used to convert the numeric gas value into text so that it can be displayed on the LCD screen.

The timer and PWM registers such as `TCCR1A`, `TCCR1B`, `ICR1`, `OCR1A`, and `OCR1B` are used to generate PWM signals for the servo motors. `OCR1A` controls the servo motor connected to the window mechanism, while `OCR1B` controls the servo motor connected to the gas valve mechanism.

In summary, the constants and variables in the project are used to define hardware connections, determine the gas threshold value, store sensor readings, display information on the LCD, and control the output devices according to the system status.

The software of this project was developed using the AVR C programming language for the ATmega328P microcontroller on the Arduino Uno board. Instead of using Arduino-specific functions, direct register-level programming was preferred in order to control the hardware components more clearly and efficiently. Registers such as DDRB, DDRD, PORTB, PORTD, ADMUX, ADCSRA, TCCR1A, TCCR1B, OCR1A, and OCR1B were used to manage input/output pins, ADC operations, and PWM signals.

The code is mainly responsible for reading the gas value from the MQ-2 gas sensor, comparing this value with the predefined threshold value, displaying the system status on the LCD screen, and controlling the warning and intervention devices. The MQ-2 sensor value is read through the A0 analog input using the ADC module of the ATmega328P. The analog sensor signal is converted into a digital value between 0 and 1023. If the measured value is below the threshold, the system stays in safe mode. In this mode, the buzzer and LED remain off, the gas valve stays open, and the window or ventilation mechanism stays closed.

When the gas value reaches or exceeds the defined threshold value, the system switches to danger mode. In this mode, the LED is turned on and the buzzer generates an audible warning. At the same time, one servo motor closes the gas valve, while the other servo motor opens the window or ventilation mechanism. The LCD screen displays a gas leakage warning message, and the gas value is also sent to the Wokwi Serial Monitor through UART communication for observation during simulation.

The code is organized into separate functions for better readability and easier control. LCD-related functions handle LCD initialization, command sending, character writing, cursor positioning, and text printing. ADC functions are used to initialize the ADC unit and read the gas sensor value. Servo functions configure Timer1 for PWM generation and control the valve and window servo positions.

UART functions are used to print sensor values and system status messages on the Wokwi Serial Monitor.

The main loop of the program continuously reads the gas sensor value and updates the system status every second. This allows the system to monitor the environment regularly and respond automatically when a dangerous gas level is detected. In this way, the code supports the main goal of the project by enabling early detection, effective warning, and autonomous physical intervention.

2. AVR C Code

```
#define F_CPU 16000000UL // 16 MHz CPU frequency because ATmega operates at this speed
```

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>
#include <stdlib.h>
```

```
#define GAS_LIMIT 400
```

```
#define BUZZER PB0 // Arduino D8, PORTB
#define LED PD7 // Arduino D7, PORTD
```

```
// LCD pins
#define LCD_RS PB4 // Arduino D12, PORTB
#define LCD_EN PB3 // Arduino D11, PORTB
```

```
// LCD 4-bit data pins
#define LCD_D4 PD5 // Arduino D5, PORTD
#define LCD_D5 PD4 // Arduino D4
#define LCD_D6 PD3 // Arduino D3
#define LCD_D7 PD2 // Arduino D2
```

```
void lcd_enable() {
    PORTB |= (1 << LCD_EN); // Set Enable pin to 1 without changing other bits
    _delay_us(1);           // Data is ready on the data pins, LCD reads it

    PORTB &= ~(1 << LCD_EN); // Set Enable pin to 0, the enable signal ends
    _delay_us(100);          // Wait for the LCD to process the received data
}
```



```

void lcd_send(int data) {
    if (data & 1) PORTD |= (1 << LCD_D4);
    else PORTD &= ~(1 << LCD_D4);

    if (data & 2) PORTD |= (1 << LCD_D5);
    else PORTD &= ~(1 << LCD_D5);

    if (data & 4) PORTD |= (1 << LCD_D6);
    else PORTD &= ~(1 << LCD_D6);

    if (data & 8) PORTD |= (1 << LCD_D7);
    else PORTD &= ~(1 << LCD_D7);

    lcd_enable();
}

// Sends a command to the LCD
void lcd_control(int command) {
    PORTB &= ~(1 << LCD_RS); // RS = 0 means command mode
    lcd_send(command >> 4); // Send upper 4 bits
    lcd_send(command & 0x0F); // Send lower 4 bits
    _delay_ms(2);
}

// Sends a character to the LCD
void lcd_character(char data) {
    PORTB |= (1 << LCD_RS); // RS = 1 means data/character mode
    lcd_send(data >> 4); // Send upper 4 bits
    lcd_send(data & 0x0F); // Send lower 4 bits
    _delay_ms(1);
}

// Initializes the LCD
void lcd_start() {
    DDRB |= (1 << LCD_RS) | (1 << LCD_EN); // Set RS and EN pins as output
    DDRD |= (1 << LCD_D4) | (1 << LCD_D5) | (1 << LCD_D6) | (1 << LCD_D7);
    // Setting DDRD bits to 1 makes these pins output pins, so data can be sent to the
    LCD

    _delay_ms(50); // Wait for LCD to become ready after power-up

    lcd_send(0x03); // 0x03 is used during LCD initialization
    _delay_ms(5);
    lcd_send(0x03); // Send again to make initialization more reliable
    _delay_ms(1);
}

```

```

    lcd_send(0x02);      // Set LCD to 4-bit mode

    lcd_control(0x28);    // 0x28 means 4-bit mode and 2-line LCD
    lcd_control(0x0C);    // 0x0C means display ON
    lcd_control(0x06);    // 0x06 means cursor moves to the right
    lcd_control(0x01);    // 0x01 means clear display
}

// Clears the LCD screen
void lcd_clear() {
    lcd_control(0x01);
    _delay_ms(2);
}

// Sets the row and column position on the LCD
// row = 0 means first row, row = 1 means second row
void lcd_set_cursor(unsigned char row, unsigned char col) {
    if (row == 0)
        lcd_control(0x80 + col); // 0x80 is the starting address of the first row
    else
        lcd_control(0xC0 + col); // 0xC0 is the starting address of the second row
}

// Prints text on the LCD
void lcd_print(char *text) {
    while (*text) {
        lcd_character(*text);
        text++;
    }
}

// Initializes the Analog to Digital Converter unit
// ADC converts the analog sensor value into a digital number
void adc_initialize() {
    ADMUX = (1 << REFS0); // ADC setting register, selects the reference voltage
    // AVcc is selected as the reference voltage

    // ADC control register
    ADCSRA = (1 << ADEN) | // Enable ADC
              (1 << ADPS2) | // Prescaler bit
              (1 << ADPS1) | // Prescaler bit
              (1 << ADPS0); // Prescaler bit
}

// Reads the MQ-2 sensor value from A0 pin
int read_gas_sensor() {

```

```

    ADMUX = (ADMUX & 0xF0) | 0;    // Select A0 channel

    ADCSRA |= (1 << ADSC);        // Start ADC conversion

    while (ADCSRA & (1 << ADSC)); // Wait until conversion is finished

    return ADC;                    // Return a value between 0 and 1023
}

// Timer1 PWM setting for servo motors
// D9 and D10 pins are used as servo outputs
void servo_initialize() {
    DDRB |= (1 << PB1) | (1 << PB2); // Set D9 and D10 as output

    TCCR1A = (1 << COM1A1) |
              (1 << COM1B1) |
              (1 << WGM11);

    TCCR1B = (1 << WGM13) |
              (1 << WGM12) |
              (1 << CS11);

    ICR1 = 20000;                  // PWM period for servo motor
}

// Valve open position
void servo_valve_open() {
    OCR1B = 1000;                  // D10
}

// Valve closed position
void servo_valve_close() {
    OCR1B = 2000;                  // D10
}

// Window closed position
void servo_window_close() {
    OCR1A = 1000;                  // D9
}

// Window open position
void servo_window_open() {
    OCR1A = 2000;                  // D9
}

```

```

// UART initialization for Wokwi Serial Monitor
void uart_initialize() {
    unsigned int ubrr = 103;    // For 9600 baud rate, F_CPU = 16 MHz

    UBRR0H = (unsigned char)(ubrr >> 8);
    UBRR0L = (unsigned char)ubrr;

    UCSROB = (1 << TXEN0);    // Enable only data transmission
    UCSROC = (1 << UCSZ01) | (1 << UCSZ00); // 8-bit data
}

// Sends one character to Serial Monitor
void uart_send_char(char data) {
    while (!(UCSROA & (1 << UDRE0)));
    UDR0 = data;
}

// Sends text to Serial Monitor
void uart_print(char *text) {
    while (*text) {
        uart_send_char(*text);
        text++;
    }
}

// Sends an integer number to Serial Monitor
void uart_print_int(int number) {
    char text[10];
    itoa(number, text, 10);
    uart_print(text);
}

// Main program
int main(void) {
    int gas_value;
    char gas_text[10];

    // Set LED and buzzer pins as output
    DDRB |= (1 << BUZZER);
    DDRD |= (1 << LED);

    adc_initialize();
    lcd_start();
    servo_initialize();
    uart_initialize();
}

```

```

// Start the system in safe mode
servo_valve_open();
servo_window_close();

lcd_clear();
lcd_set_cursor(0, 0);
lcd_print("Gas System");
lcd_set_cursor(1, 0);
lcd_print("Starting...");
_delay_ms(2000);

while (1) {
    // Read value from gas sensor
    gas_value = read_gas_sensor();

    uart_print("Gas Value: ");
    uart_print_int(gas_value);
    uart_print("\r\n");

    // Convert number to text
    itoa(gas_value, gas_text, 10);

    // Display gas value on LCD
    lcd_clear();
    lcd_set_cursor(0, 0);
    lcd_print("Gas:");
    lcd_print(gas_text);

    // If gas value exceeds threshold, danger mode runs
    if (gas_value >= GAS_LIMIT) {
        PORTB |= (1 << BUZZER); // Turn buzzer on
        PORTD |= (1 << LED);    // Turn LED on

        servo_valve_close();    // Close gas valve
        servo_window_open();    // Open window

        lcd_set_cursor(1, 0);
        lcd_print("GAS LEAKAGE!");

        uart_print("- GAS LEAKAGE DETECTED\r\n");

        int i;
        for (i = 0; i < 200; i++) {
            PORTB |= (1 << BUZZER); // Buzzer HIGH
            _delay_us(500);
        }
    }
}

```

```

        PORTB &= ~(1 << BUZZER); // Buzzer LOW
        _delay_us(500);
    }
}

// If gas value is low, normal mode runs
else {
    PORTB &= ~(1 << BUZZER); // Turn buzzer off
    PORTD &= ~(1 << LED);    // Turn LED off

    servo_valve_open();    // Gas valve open
    servo_window_close();  // Window closed

    lcd_set_cursor(1, 0);
    lcd_print("Status: Safe");

    uart_print("- SAFE\r\n");
}

// New measurement is taken every second
_delay_ms(1000);
}

return 0;
}

```

3. WOKWI diagram.json File

```

{
  "version": 1,
  "author": "Anonymous maker",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-breadboard-mini", "id": "bb1",
    "top": 17.8, "left": -31.2, "attrs": {} },
    { "type": "wokwi-arduino-uno", "id": "uno", "top":
365.4, "left": 47.4, "attrs": {} },
    { "type": "wokwi-gas-sensor", "id": "gas1", "top":
348.3, "left": -165.8, "attrs": {} },
    {
      "type": "wokwi-buzzer",
      "id": "bz1",
      "top": 108,
      "left": -94.2,
      "attrs": { "volume": "0.1" }
    },
    { "type": "wokwi-lcd1602", "id": "lcd1", "top":
157.03, "left": 409.6, "attrs": {} },

```

```

    { "type": "wokwi-led", "id": "led1", "top": 6,
"left": 32.6, "attrs": { "color": "red" } },
    { "type": "wokwi-servo", "id": "servo1", "top": 46,
"left": 441.6, "attrs": {} },
    {
        "type": "wokwi-resistor",
        "id": "r1",
        "top": 51.95,
        "left": -9.6,
        "attrs": { "value": "220000" }
    },
    { "type": "wokwi-servo", "id": "servo2", "top":
-30.8, "left": 441.6, "attrs": {} }
],
    "connections": [
        [ "gas1:VCC", "uno:5V", "red", [ "h38.4", "v181.5",
"h172.8" ] ],
        [ "gas1:GND", "uno:GND.2", "black", [ "h28.8",
"v200.8", "h192" ] ],
        [ "gas1:A0UT", "uno:A0", "green", [ "h19.2",
"v230.4", "h240" ] ],
        [ "bz1:2", "uno:8", "green", [ "v57.6", "h95.6" ] ],
        [ "bb1:9t.e", "uno:7", "green", [ "v144",
"h-9.6" ] ],
        [ "bz1:1", "uno:GND.1", "black", [ "v76.8", "h230.1"
] ],
        [ "bb1:2t.e", "uno:GND.1", "green", [ "v153.6",
"h124.5" ] ],
        [ "servo1:GND", "uno:GND.1", "black", [ "h0" ] ],
        [ "servo1:V+", "uno:5V", "red", [ "h-115.2",
"v460.9", "h-119" ] ],
        [ "lcd1:RS", "uno:12", "green", [ "v9.6",
"h-287.7" ] ],
        [ "lcd1:E", "uno:11", "green", [ "v19.2",
"h-297.6" ] ],
        [ "lcd1:D4", "uno:5", "green", [ "v28.8",
"h-278.4" ] ],
        [ "lcd1:D5", "uno:4", "green", [ "v38.4",
"h-278.3" ] ],
        [ "lcd1:D6", "uno:3", "green", [ "v48",
"h-278.2" ] ],
        [ "lcd1:D7", "uno:2", "green", [ "v57.6",
"h-278.1" ] ],
        [ "lcd1:VSS", "uno:GND.1", "black", [ "v0",
"h-278.4" ] ],
        [ "lcd1:VDD", "uno:5V", "red", [ "v288", "h-239.9" ]
],
        [ "lcd1:RW", "uno:GND.3", "black", [ "v297.6",
"h-259.3" ] ],

```

```

    [ "lcd1:A", "uno:5V", "red", [ "v316.8", "h-374.5" ]
  ],
  [ "lcd1:K", "uno:GND.3", "black", [ "v326.4",
"h-355.2" ] ],
  [ "servo2:PWM", "uno:10", "orange", [ "h0" ] ],
  [ "servo2:V+", "uno:5V", "red", [ "h-96", "v547.3",
"h-134.4" ] ],
  [ "servo2:GND", "uno:GND.1", "black", [ "h-192",
"v259.2", "h-86.4" ] ],
  [ "led1:A", "bb1:9t.c", "", [ "$bb" ] ],
  [ "led1:C", "bb1:8t.c", "", [ "$bb" ] ],
  [ "r1:1", "bb1:2t.d", "", [ "$bb" ] ],
  [ "r1:2", "bb1:8t.d", "", [ "$bb" ] ],
  [ "servo1:PWM", "uno:9", "orange", [ "h0" ] ]
],
"dependencies": {}
}

```

4. WOKWI libraries.txt File

Wokwi Library List

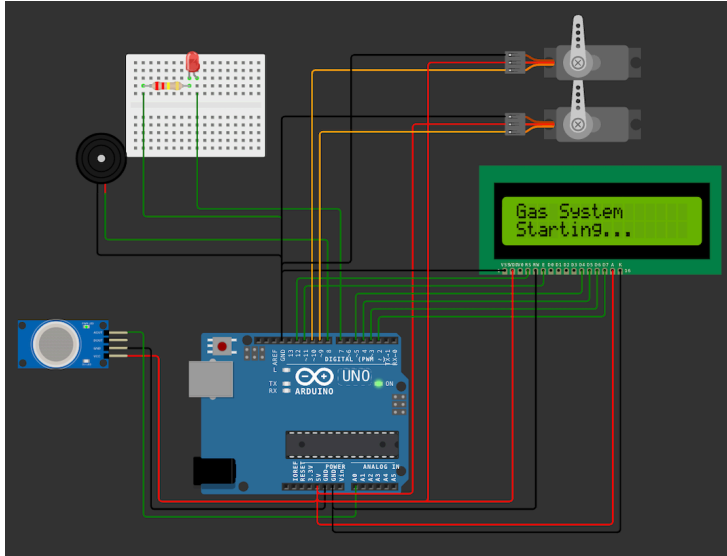
See <https://docs.wokwi.com/guides/libraries>

Keypad

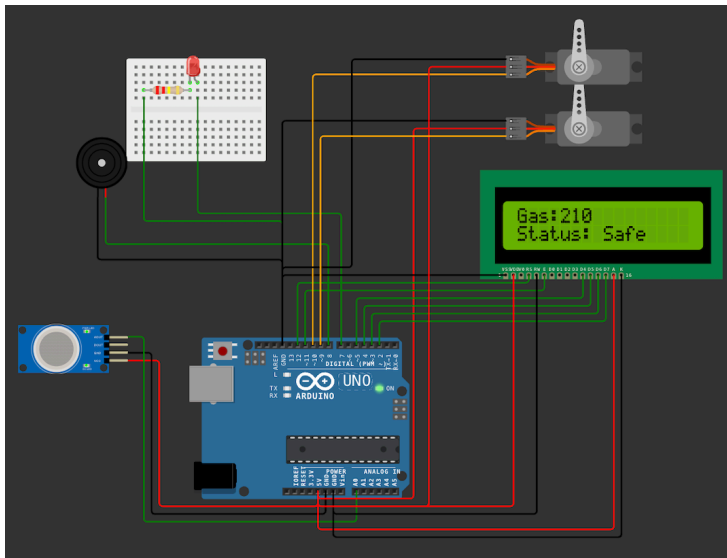
LiquidCrystal

Servo

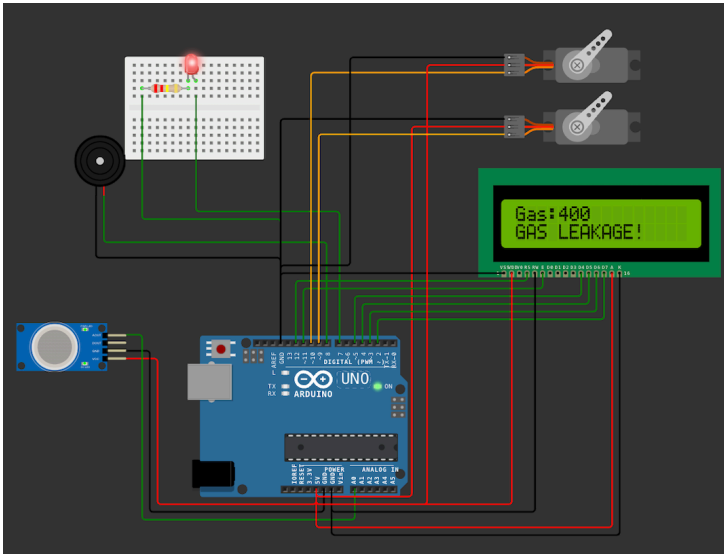
6. CHAPTER: PROJECT RUNTIME SCREENSHOT IMAGES



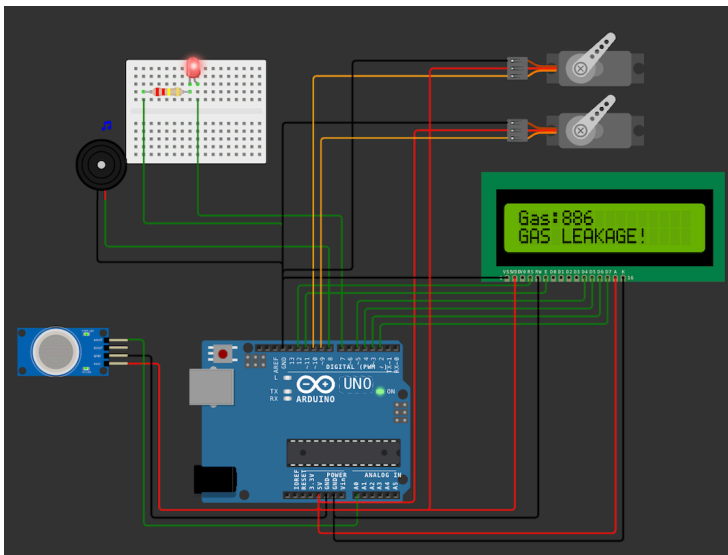
This image shows the initial state of the gas leakage detection system. The LCD displays “Gas System Starting...”, which means that the system is being initialized. At this stage, the MQ-2 gas sensor, LCD screen, LED, buzzer, and servo motors are being prepared for operation. The gas valve servo and window servo are in their default safe positions.



This image shows the system operating in safe mode. The MQ-2 gas sensor value is below the predefined threshold value. The LCD displays the measured gas value and the message “Status: Safe”. Since no gas leakage is detected, the LED and buzzer remain off, the gas valve stays open, and the window or ventilation servo remains closed.



This image shows the system switching to danger mode when the gas value reaches the threshold value. The LCD displays “Gas: 400” and “GAS LEAKAGE!”, indicating that the detected gas level has reached the predefined gas limit. In this case, the warning system is activated, the LED turns on, the buzzer gives an audible alarm, the gas valve servo closes the valve, and the window servo opens the ventilation mechanism.



This image shows the system response when the gas concentration is significantly above the threshold value. The LCD displays “Gas: 886” and “GAS LEAKAGE!”. This indicates a dangerous gas leakage condition. The system automatically activates the alarm and mitigation mechanisms by turning on the LED, activating the buzzer, closing the gas valve, and opening the window or ventilation system through the servo motors.

7. CHAPTER: CONCLUSIONS AND FUTURE WORK

In conclusion, the presented project, “**Gas Leakage Detection and Automatic Control System**” demonstrates that an embedded system can be effectively used to detect gas leakage and automatically respond to dangerous situations. The project is designed to reduce the risks caused by natural gas, LPG, and similar hazardous gases in residential and industrial environments. Unlike traditional alarm systems that only warn the user, this system also performs autonomous physical intervention by closing the gas valve and opening the window or ventilation mechanism.

Within the scope of the project, an Arduino Uno board with the ATmega328P microcontroller was used as the main control unit. The MQ-2 gas sensor was used to measure the gas concentration in the environment. The analog value received from the sensor was converted into a digital value by the ADC module of the ATmega328P and compared with a predefined threshold value. According to this comparison, the system decided whether the environment was safe or dangerous.

The system was developed using the AVR C programming language. Instead of using Arduino-specific libraries and ready-made functions, direct register-level programming was used to control the hardware components. LCD control, ADC reading, buzzer and LED control, servo motor movement, PWM generation, and UART communication were implemented through AVR registers. This approach provided a clearer understanding of how the microcontroller communicates with each hardware component.

The project includes several hardware components such as an MQ-2 gas sensor, 16x2 LCD screen, LED, buzzer, and two servo motors. The LCD screen displays the current gas value and system status. In safe mode, the system shows that the gas level is normal, keeps the buzzer and LED off, leaves the gas valve open, and keeps the window closed. When the gas value reaches or exceeds the defined threshold value, the system switches to danger mode. In this mode, the LED turns on, the buzzer gives an audible warning, the gas valve servo closes the valve, and the

window servo opens the ventilation mechanism. The system status and gas value are also displayed through the Wokwi Serial Monitor using UART communication.

The simulation results show that the system responds correctly under different gas concentration levels. When the gas value is below the threshold, the system remains in safe mode. When the value reaches or exceeds the threshold, the alarm and mitigation mechanisms are activated automatically. This proves that the designed embedded system can detect a possible gas leakage condition and respond without waiting for human intervention.

One of the main advantages of this project is its practical use in emergency situations. In real gas leakage events, users may be asleep, away from the environment, panicked, or unable to react quickly. Therefore, a system that can automatically detect the danger and take action provides an important safety improvement. The project can be adapted for homes, factories, warehouses, industrial kitchens, laboratories, mines, and similar areas where gas leakage risk exists.

However, the current version of the project has some limitations. The system was tested in the Wokwi simulation environment, so real-life environmental conditions such as sensor calibration, gas distribution, temperature, humidity, and hardware response time were not fully evaluated. Also, the MQ-2 sensor provides general gas detection and may require calibration for more accurate measurements in real applications. In addition, the servo motor mechanism used in the simulation represents the gas valve and ventilation system conceptually.

For future work, the system can be improved by adding more advanced sensors, such as a dedicated CO sensor, temperature and humidity sensor, or flame sensor. Wireless communication modules such as Wi-Fi, GSM, or Bluetooth can also be added to send notifications to users remotely. A mobile application or IoT dashboard can be developed to monitor gas levels in real time. In addition, real hardware implementation can be carried out to test the system under actual

environmental conditions. Battery backup and fail-safe mechanisms can also be added to make the system more reliable during power outages.

As a result, this project presents a practical, low-cost, and autonomous embedded safety solution for gas leakage detection and mitigation. By combining sensor-based monitoring, visual and audible warnings, and automatic physical intervention, the system aims to reduce the risk of poisoning, fire, explosion, and property damage. The project successfully demonstrates how embedded systems can be used in real-life safety applications and provides a strong foundation for further development.

8. REFERENCES

Yıldız, Ş. (2011). Karbonmonoksit Zehirlenmesi ve Türkiye Verileri. IV. Ulusal Sualtı Hekimliği ve Hiperbarik Tıp Kurultayı, 29-30 Nisan 2011, GATA Haydarpaşa Eğitim Hastanesi, İstanbul. <https://sualti.org/wp-content/uploads/2016/02/HITAM-4.pdf#page=35>

Yıldırım, G., Acar, B., & Gençel, O. (2015). GSM ve GPRS Haberleşmeli Merkezi Gaz Seviye Kontrol Sistemi. *Akademik Bilişim Konferansı (AB'15)*, İstanbul Teknik Üniversitesi. <https://www.ab.org.tr/inetd.org.tr/ab15/bildiri/117.pdf>

Robotistan. (2024). Elektronik Bileşen Fiyatları. <https://www.robotistan.com>

Robot Sepeti. (2024). MQ-2 Yanıcı Gaz ve Duman Algılayıcı Sensörü Modülü. <https://www.robotsepeti.com/mq-2-yanici-gaz-ve-duman-algilayici-sensoru-modulu>